

Raritone Backend Architecture, Media Workflow, Security & Scaling Documentation

Meka Hrishi Teja Chowdary

1. Project Overview

Raritone is an AI-powered virtual try-on platform designed to provide real-time clothing visualization, AI-based avatar support, and scalable backend infrastructure.

The backend system is developed using:

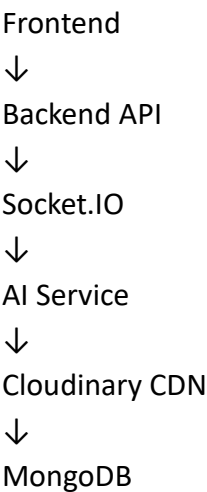
- Node.js
- Express.js
- MongoDB
- Socket.IO
- Cloudinary CDN

The architecture is designed to support:

- Realtime communication
- AI image processing
- Optimized media delivery
- Scalable APIs
- Secure backend workflows

2. Backend Architecture

Architecture Flow





Response

Backend Components

Frontend

Handles:

- User interactions
- Image uploads
- Virtual try-on requests
- Realtime updates

Express Backend API

Responsible for:

- API routing
- Authentication
- Validation
- Business logic
- Media processing

Socket.IO Layer

Provides:

- Realtime communication
- Live try-on updates
- Instant response delivery

AI Service Layer

Planned for:

- Virtual try-on processing
- Avatar generation
- Recommendation engine
- AI image processing

Cloudinary CDN

Handles:

- Image storage
- Optimized delivery
- Image transformation
- CDN acceleration

MongoDB Database

Stores:

- Users
- Products
- Try-on history
- Measurements
- Avatar data

3. Media Workflow

Media Upload Flow

User Uploads Image

↓

Multer Upload Handling

↓

Sharp Compression

↓

Cloudinary Upload

↓

Optimized CDN URL

↓

MongoDB Storage

↓

Frontend Delivery

Multer Upload Flow

Multer middleware is used for:

- Receiving uploaded images
- Handling multipart form-data
- Temporary memory storage

This enables efficient backend image handling.

Sharp Compression

Sharp is used for:

- Image resizing
- Image compression
- Optimized image processing

Example optimizations:

- Reduced image dimensions
- Reduced file size
- Improved loading speed

Benefits:

- Lower bandwidth usage
- Faster API responses
- Better frontend performance

Cloudinary Upload

Compressed images are uploaded to Cloudinary CDN.

Cloudinary provides:

- Scalable cloud storage
- Optimized media delivery
- Automatic format optimization
- Secure media management

CDN Optimization

Optimized URLs use:

- w_500
- h_500
- c_fill
- f_auto
- q_auto

Benefits:

- Faster loading
- Smaller payloads
- Modern image formats
- Improved scalability

Optimized Image Delivery

Frontend receives:

- Compressed images
- Optimized CDN URLs
- Faster image rendering

This improves:

- User experience
- Frontend performance
- API efficiency

Cleanup Handling

If upload fails:

- Unused Cloudinary images are deleted
- Invalid media data is cleaned automatically

This prevents:

- Storage waste
- Orphan media files

4. Security & Performance

JWT Protection

JWT authentication is implemented for:

- Secure login
- Protected APIs
- User authorization

Protected routes require valid tokens before access.

Joi Validation

Joi validation is used for:

- Login validation
- Registration validation
- Product validation
- Try-on validation

Benefits:

- Prevents invalid requests
- Improves API reliability
- Protects backend data integrity

Helmet Security

Helmet middleware adds secure HTTP headers.

Protection includes:

- Content security policies
- Clickjacking prevention
- MIME-type protection

Morgan Logging

Morgan is used for:

- API request logging
- Debugging
- Request monitoring

This improves:

- Backend monitoring
- Debugging efficiency

Rate Limiting

Rate limiting protects APIs from:

- Spam requests
- Brute force attacks

- Excessive API traffic

This improves backend stability and security.

Error Middleware

Centralized error middleware handles:

- API errors
- Validation failures
- Server exceptions

Benefits:

- Consistent responses
- Easier debugging
- Better API reliability

Compression Optimization

Compression middleware reduces:

- API payload sizes
- Response times
- Bandwidth usage

This improves:

- Scalability
- Frontend speed
- API performance

5. Realtime Communication

Socket.IO enables:

- Realtime try-on updates
- Live status communication
- Instant event delivery

Example Workflow

Frontend Sends Try-On Request

↓

Backend Receives Event

↓

AI Processing Starts

↓

Realtime Status Updates Sent

↓

Frontend Receives Final Result

Benefits:

- Reduced API polling
- Faster user experience
- Realtime interaction support

6. Scaling Research

Scaling For 10,000+ Users

Required optimizations:

- MongoDB indexing
- Optimized queries
- Image compression
- CDN optimization
- API caching

Current implementation already includes:

- Compression
- Optimized image delivery
- CDN optimization
- Lean database queries

Scaling For 100,000+ Users

Future scaling requirements include:

Redis Caching

Used for:

- API caching
- Session caching

- Faster repeated requests

Load Balancing

Distributes traffic across multiple servers.

Benefits:

- Reduced server overload
- Improved uptime
- Better scalability

Queue Systems

Used for:

- AI processing queues
- Async image processing
- Background jobs

Examples:

- BullMQ
- RabbitMQ

Microservices Architecture

Separates:

- Authentication
- AI processing
- Media services
- Realtime services

Benefits:

- Independent scaling
- Improved maintainability
- Better performance

AI Scaling

Future AI infrastructure may require:

- GPU servers

- Distributed processing
- AI task queues

7. Monitoring Research

Sentry

Used for:

- Error tracking
- Crash monitoring
- Backend debugging

Benefits:

- Realtime error alerts
- Production monitoring

Grafana

Used for:

- Server monitoring
- Performance analytics
- Infrastructure visualization

Benefits:

- Resource monitoring
- System analytics

Firebase Analytics

Used for:

- User analytics
- Usage tracking
- Engagement monitoring

Benefits:

- User behavior insights
- Product improvement analysis

8. Progress Report

Completed Features

Implemented:

- JWT authentication
- Google OAuth
- Product APIs
- Wishlist APIs
- Cart APIs
- Measurement APIs
- Realtime Socket.IO setup
- Cloudinary integration
- Sharp compression
- Joi validation
- Helmet security
- Morgan logging
- Rate limiting
- Centralized error handling

Realtime APIs

Completed:

- Socket.IO integration
- Realtime communication
- Live backend event handling

Media Optimization

Completed:

- Image compression
- CDN optimization
- Optimized image delivery
- Cloudinary workflow

Security Implementation

Completed:

- JWT protection
- Helmet middleware
- Rate limiting
- Joi validation
- Secure environment variables

AI Workflow Planning

Prepared:

- AI API structure
- Avatar architecture
- Try-on workflow
- Recommendation system planning

Future Improvements

Possible future enhancements:

- AI processing queues
- Redis caching
- Distributed websocket scaling
- GPU AI infrastructure
- Microservices architecture
- Advanced monitoring systems

Conclusion

The Raritone backend architecture is designed to support scalable AI-powered virtual try-on features with optimized media workflows, realtime communication, secure APIs, and future-ready infrastructure planning.

The current backend implementation provides a strong foundation for future AI integrations, large-scale user handling, and realtime interactive experiences.